

12) Observability, Operations and Support

Last updated by | Mennell, Jacob | 19 May 2026 at 15:45 BST

Observability, Operations, And Support

Purpose

This page explains how the solution surfaces operational health, how failures are handled, and where support teams should start when diagnosing issues.

Scope

Observability is reflected through a combination of:

- API health endpoints
- application logging
- QA and retry telemetry in orchestration flows
- operational state transitions for jobs and tasks
- pipeline-level test and coverage reporting

Health Signals

The API exposes a simple health-check endpoint, and the local runtime topology also includes container health checks and dependency ordering. These are baseline liveness signals rather than full business-health diagnostics.

Operationally, the system's true health also depends on:

- worker availability
- queue health
- successful ingestion state
- availability of retrieval-ready chunk data
- successful downstream model or service calls

Logging And Runtime Diagnostics

The service code uses logging across the API and worker layers. The architecture notes also highlight telemetry around retrieval confidence, QA behavior, candidate counts, acceptance depth, and retry patterns.

These signals are important because many failures in AI-assisted systems are not binary crashes. They often appear as:

- low-confidence retrieval
- repeated QA retries
- recoverable agent failures
- fail-soft outputs such as `stuck`

Retry And Failure Behavior

The orchestration architecture deliberately includes bounded retry layers rather than unlimited self-retry behavior. This improves operational predictability.

Examples include:

- transient retry around runner or infrastructure failures
- constrained local retry within execution nodes
- graph-level retry loops controlled by QA policy
- terminal fail-safe behavior when progressive-stop does not yield an accepted attempt

This design helps support teams distinguish between transient recoverable issues and persistent logic or evidence problems.

Support Entry Points

When troubleshooting, the most useful starting points are typically:

1. confirm the API and worker containers are healthy
2. confirm the document was ingested successfully and chunk data exists
3. confirm background jobs were queued and not revoked or left stale
4. inspect retrieval quality and QA outcomes for the task
5. inspect feedback or user-facing evidence views for traceability gaps

This sequence aligns with the actual architecture: ingest first, retrieve second, reason third, deliver fourth.

Operational Signals Worth Tracking

Based on the architecture notes, the following are especially useful:

- QA pass rate by attempt
- retry counts and retry route distribution
- progressive-stop acceptance depth
- constrained retry invocation rate in checks
- percentage of fail-soft stuck outcomes
- retrieval confidence signals and candidate counts

These measures help separate prompt/model issues from retrieval issues and infrastructure issues.

Operations Diagram

flowchart TD

Health[Health Check and Service Status] --> Ingest[Verify Ingestion Success]

Ingest --> Queue[Verify Job and Queue State]

Queue --> Retrieve[Inspect Retrieval and QA Signals]

Retrieve --> Output[Review User-Facing Output and Sources]



Design Decisions

- The system prefers bounded retry over uncontrolled retry loops.
- QA is treated as an observable control point rather than hidden inside execution.

- Fail-soft outputs are used to preserve workflow continuity and improve diagnosability.
- Operational state is reflected in both runtime logs and persisted workflow/task records.